Virtual Try-On Pipeline

Team workflow, repository structure, and integration plan for the AI/ML internship project

Project: AI-powered virtual try-on system using the DeepFashion2 dataset. The pipeline takes a person image and a garment image, detects body pose, segments existing clothing, warps the new garment to match body posture, and blends the result into a realistic try-on output.

Repository	github.com/pavansaivenna-maker/virtual-tryon-pipeline
Team size	4 (1 AI Lead + 3 specialist interns)
Modules	4 (Dataset, Segmentation, Warping, Blending)
Tech stack	Python, PyTorch, FastAPI, OpenCV, MediaPipe, SAM, HR-VITON
Branching	main → dev → 4 feature branches
Cadence	Daily standup, Wednesday integration, Friday demo

1. Team and Responsibilities

Each intern owns one module of the pipeline end-to-end — implementation, testing, and the API service that exposes their work to the next stage. The AI Lead owns the integration layer that glues these modules together.

Role	Module	Primary responsibilities
AI Lead	Module 4 — Blending & Integration	Pipeline architecture, alpha blending, API orchestration, integration testing,
Data Intern	Module 1 — Dataset Preparation	Loading DeepFashion2, cleaning corrupted records, image resizing and nor
CV Intern	Module 2 — Parsing & Segmentation	Pose detection (MediaPipe), cloth segmentation (SAM / U-2-Net), agnostic p
ML Intern	Module 3 — Garment Warping	TPS transformation implementation, geometric matching, warp confidence s

2. Pipeline Architecture

The system is built as four sequential modules connected by FastAPI services. Each module receives a well-defined input from the previous stage and produces a well-defined output for the next. This contract-first approach allows all four interns to work in parallel without breaking each other's code.

Data flow

```
Person image + Garment image
    ↓
[Module 1] Dataset prep              ←  Data intern
    ↓ (cleaned, structured dataset)
[Module 2] Parsing & segmentation  ←  CV intern
    ↓ (pose keypoints, masks, agnostic image)
[Module 3] Garment warping (TPS)   ←  ML intern
    ↓ (warped garment with alpha)
[Module 4] Blending + APIs         ←  AI Lead
    ↓
Final try-on image
```

3. Repository Structure

Each intern works exclusively in their own folder, eliminating merge conflicts. Shared files (contracts, schemas, root configuration) require a PR reviewed by all team members.

```
virtual-tryon-pipeline/
■■■ README.md                  # Project overview, setup, workflow rules
■■■ contracts.md             # Data formats between modules (READ FIRST)
■■■ requirements.txt         # Shared Python dependencies
■
■■■ data/                    # Data intern's workspace
■   ■■■ raw/               # DeepFashion2 raw data (gitignored)
■   ■■■ processed/         # Cleaned, resized images + JSON
■   ■■■ prepare.py         # Dataset prep script
■
■■■ services/
■   ■■■ segmentation/      # CV intern → POST /ai/segment-cloth
■   ■■■ warping/           # ML intern → POST /ai/warp-garment
■   ■■■ tryon/             # AI Lead → POST /ai/tryon
■
■■■ shared/
■   ■■■ schemas.py         # Pydantic models matching contracts.md
■
■■■ samples/                # 5–10 reference images everyone tests against
■
■■■ tests/
    ■■■ integration/      # End-to-end pipeline tests (AI Lead)
```

4. Branching Strategy

Six branches total. Feature branches are owned by individual interns. All merges into `dev` require a Pull Request reviewed by the AI Lead. Only the AI Lead merges `dev` into `main`.

Branch	Owner	Purpose
main	AI Lead	Production-ready, releasable code only
dev	AI Lead	Integration branch — feature branches merge here first
feature/data	Data intern	Dataset preparation work
feature/segmentation	CV intern	Parsing, masks, agnostic image
feature/warping	ML intern	TPS warping engine
feature/integration	AI Lead	Blending, API orchestration, tests

Workflow rules

- Never push directly to `main` or `dev`
- Always open Pull Requests into `dev`
- Tag the AI Lead as reviewer on every PR
- Delete merged feature branches locally before starting new work

- Any change to `contracts.md` or `shared/schemas.py` requires review from all four team members

5. Data Contracts (Module Handoffs)

The contract file is the most important document in the project. It defines the exact format of every handoff between modules. If a module's output doesn't match the contract, the pipeline is broken. The contract takes precedence over implementation convenience — if a model can't produce the contracted output, the team decides whether to update the model or update the contract (and bump the version).

Global image conventions

All images in the pipeline are **512 × 384 pixels**, portrait orientation, RGB color space. Masks use single-channel PNG. Filenames follow the pattern `{type}_{id:04d}.{ext}` (e.g. `person_0001.jpg`, `mask_0042.png`).

Handoff 1 — Data → CV

Data intern produces `data/processed/` containing:

- `images/` — 512×384 RGB person photos
- `garments/` — 512×384 RGB garment photos with white background
- `annotations.json` — per-image metadata (image path, garment bbox, category, original size)
- `splits.json` — train/val/test image ID lists

Handoff 2 — CV → ML

CV intern's segmentation service returns:

- **18 pose keypoints** in COCO format, each `[x, y, confidence]`
- **Segmentation mask** — single-channel PNG with labels: 0=bg, 1=upper, 2=lower, 3=skin, 4=face, 5=hair
- **Agnostic person image** — RGB JPG with garment region replaced by neutral gray (128,128,128)
- **Confidence score** — float between 0 and 1

Handoff 3 — ML → AI Lead

ML intern's warping service returns:

- **Warped garment** — RGBA PNG, 512×384, alpha channel marks garment region
- **Warp confidence** — float 0–1, values below 0.5 mean the warp is unreliable
- **Warp method** — one of `"tps"`, `"gmm"`, `"flow"` (useful for debugging)

Handoff 4 — AI Lead → Final Output

The try-on service composites the warped garment onto the agnostic image using alpha blending: `I_final = α · F_warped + (1 - α) · B_agnostic`. Output is RGB JPG, 512×384, with face/hair/skin preserved from the original person image.

6. API Endpoints

Each module exposes its functionality through a FastAPI service. This decouples modules at runtime — each intern can develop and test independently, and integration failures get isolated to the API boundary instead of buried in shared notebooks.

Endpoint	Owner	Purpose
POST /ai/segment-cloth	CV intern	Accepts a person image. Returns pose keypoints, segmentation mask path, agn
POST /ai/warp-garment	ML intern	Accepts garment image, pose keypoints JSON, and segmentation mask. Return
POST /ai/tryon	AI Lead	Public endpoint. Accepts person and garment images. Internally calls segment-

Running the pipeline locally

```
# Terminal 1 — segmentation service
cd services/segmentation && uvicorn app:app --port 8001

# Terminal 2 — warping service
cd services/warping && uvicorn app:app --port 8002

# Terminal 3 — try-on service (calls the other two)
cd services/tryon && uvicorn app:app --port 8003

# Test with curl
curl -X POST http://localhost:8003/ai/tryon \
  -F "person=@samples/person_001.jpg" \
  -F "garment=@samples/garment_001.jpg" \
  -o output.png
```

7. Setup Steps Completed

The following setup work has been completed by the AI Lead. The project is ready for kickoff.

#	Step	Status
1	GitHub repository created (private)	Done
2	Team chat channel established	Done
3	Folder structure created and pushed	Done
4	README.md and contracts.md written	Done
5	Six branches created (main, dev, 4 feature branches)	Done
6	GitHub Projects board with 12 labeled, assigned tasks	Done
7	Kickoff call with all interns	Pending
8	First end-to-end integration run	Pending

8. Sprint Tasks (Issue Board)

Twelve issues were created and assigned across the team. The board uses four labels color-coded by owner: **data** (green), **cv** (purple), **ml** (orange), **integration** (blue).

#	Title	Label
1	Set up DeepFashion2 dataset download	data
2	Build dataset preparation script	data
3	Generate annotations.json and splits.json	data
4	Implement pose detection with MediaPipe	cv
5	Implement cloth segmentation	cv
6	Generate agnostic person image	cv
7	Wrap segmentation in FastAPI service	cv
8	Implement TPS warping engine	ml
9	Add warp confidence score	ml
10	Wrap warping in FastAPI service	ml
11	Implement alpha blending in try-on service	integration
12	Build end-to-end integration tests	integration

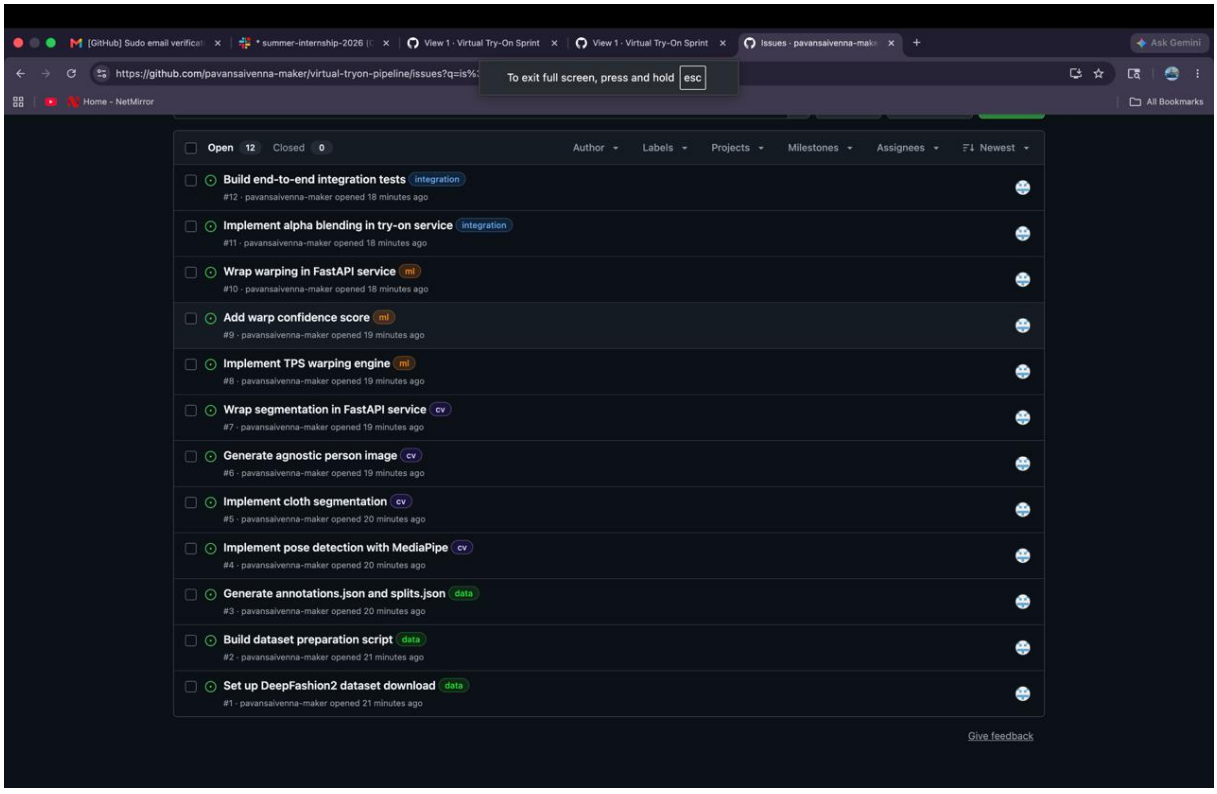

Figure 1 — GitHub Issues board with all 12 tasks created, labeled by role (data / cv / ml / integration), and assigned to the AI Lead pending intern collaborator invites.

9. Team Cadence

Three rituals — kept light to avoid meeting fatigue while staying coordinated.

When	What	Duration	Format
Daily, 10:00 AM	Text standup: yesterday / today / blockers	5 min each	Async chat
Wednesday afternoon	Mid-sprint integration run by AI Lead	1 hour	AI Lead solo, posts results
Friday end of day	Sprint demo + retrospective	30 min	Video call

10. Wednesday Integration Workflow

Every Wednesday the AI Lead runs an end-to-end integration test against the current state of all four branches. This catches contract mismatches early, while it's still cheap to fix them.

```
cd ~/virtual-tryon-pipeline
git checkout dev
git pull
git merge feature/data
git merge feature/segmentation
git merge feature/warping
git merge feature/integration

# Then run the full pipeline on a sample image
python tests/integration/run_pipeline.py samples/person_001.jpg samples/garment_001.jpg
```

What to do when things break

- For each failure, note the exact error message and which module's output didn't match the contract
- Use the `stage` field in error responses to identify the responsible module
- Post results in the team chat with the failing input image, expected vs actual output, and the owning intern tagged
- For first integrations, mocked outputs are acceptable — the goal is testing the contract, not model quality
- If a contract change is needed, bump the version in `contracts.md` and notify the team

11. AI Lead Responsibilities

Beyond owning Module 4, the AI Lead is responsible for the glue that holds the project together. These responsibilities are continuous throughout the sprint.

- **Maintain contracts.md** — review proposed changes, bump versions, broadcast updates to the team
- **Review all Pull Requests** into `dev` within 24 hours of submission
- **Run Wednesday integrations** — never skip, even when modules are incomplete
- **Maintain the Project board** — keep issue statuses current, reassign blocked work
- **Monitor team health** — DM any intern who skips two standups in a row
- **Own the FastAPI scaffolding** in `services/tryon/` and Pydantic schemas in `shared/`
- **Write integration tests** in `tests/integration/` that exercise the full pipeline
- **Manage the sprint demo** — drive the screen, narrate the pipeline, fielding questions

12. Known Risks and Mitigations

Common failure modes for student/intern AI projects, and how this workflow addresses each one.

Risk	Mitigation
Interns work in isolation for weeks, then their outputs don't connect	Wednesday integration runs starting in week 1, even with mocked outputs
Quiet intern gets stuck and stops asking for help	Daily standups visible in chat; AI Lead DMs anyone who skips two days
Contract gets violated, modules silently produce wrong outputs	Pydantic schemas validate every API request/response at runtime
Pose model produces different keypoint count than contract specifies	Version contracts; expect updates in week 1, lock by week 2
Hardware constraints — interns can't run full pipeline locally	Each FastAPI service runs independently; can be stubbed with mock responses
Last-week panic when nothing connects	End-to-end integration in week 1 prevents this entirely

13. Final Deliverables

By the end of the internship, the project will produce the following artifacts:

- Working virtual try-on pipeline running end-to-end on sample images
- Cloth segmentation system with documented accuracy on a held-out test set
- Pose estimation module producing COCO-format keypoints
- TPS garment warping engine with confidence scoring
- Final try-on output generator with alpha blending
- Three FastAPI services exposing the pipeline as REST endpoints
- End-to-end integration test suite with sample test cases
- Project documentation: README, contracts, and per-module READMEs
- Sprint retrospective notes and final demo recording

14. Closing Note

The biggest risk in this project is social, not technical. Contracts will get violated, integrations will fail repeatedly in the first two weeks, and at least one intern will go quiet for a few days. None of this is a sign the workflow isn't working — it is exactly when the workflow is doing its job. The AI Lead's role is not to prevent these problems but to surface them fast through the Wednesday integration and daily standups, and intervene before they become "one week left and nothing connects."

Build the boring scaffolding now. Run integration ugly and early. Update contracts shamelessly. Watch for the quiet intern. That is the entire job.